Covert

Incident analysis report - covert command and control (TrevorC2) and DNS exfiltration

Compromised host	Windows workstation 192.168.1.11 (host name MSEDGEWIN10 in the decrypted commands)
C2 server	192.168.1.30 (HTTP, serving a cloned web page)
Other hosts of interest	192.168.1.29 (port scan and content discovery), 192.168.1.37 (content discovery)
Technique observed	Covert HTTP command and control consistent with TrevorC2, plus DNS exfiltration to `fakec2server.domain`
Evidence analyzed	Network capture `output.pcap` (activity on 2021-09-16)
Tools used	Wireshark, CyberChef, Python (PyCryptodome), Kali Linux
Overall result	Confirmed covert C2 channel on 192.168.1.11: commands recovered and decrypted, and credential-like data sent out over DNS

1. Executive Summary

The capture shows a Windows workstation at 192.168.1.11 that was first probed by other hosts and then communicated with 192.168.1.30 through a covert command and control (C2) channel hidden inside ordinary-looking web traffic. The workstation sent HTTP GET requests whose `guid` parameter carried long Base64 strings, and the C2 server answered with a copy of a legitimate web page in which an encrypted command was hidden inside an HTML comment.

I recognised the framework from the `oldcss` comment marker and from its hard-coded passphrase, and I wrote a small Python script that reproduces its AES-CBC routine. With it I decrypted the commands the operator sent: directory listings, a move to `C:\Windows\Temp`, a marker file containing an infection identifier, and finally a PowerShell script launched with the execution policy bypassed. In the same time window the host sent DNS queries with Base64 subdomains to an invented domain; decoding them in CyberChef revealed credential-like text. The workstation must be treated as compromised.

2. Scope and Evidence

The only evidence supplied is one packet capture, `output.pcap`, opened in Wireshark. The packets I examined span 13:01 to 14:17 on 2021-09-16. CyberChef was used to decode the DNS data, and a Python script running on a Kali Linux workstation was used to decrypt the C2 values. All findings below come from that capture.

3. Background: How TrevorC2 Hides Its Traffic

TrevorC2 is an open-source covert C2 framework published by TrustedSec. Its server clones an existing web site and hides the operator's command inside the page source, in a parameter whose name can be customised. The client (written in Python and available for Windows, macOS and Linux) requests the page at random intervals, extracts and runs the command, and returns the output as Base64 data in a query-string parameter, using GET rather than POST. To a casual observer the traffic looks like a browser visiting a normal site.

In this capture the hidden values are also encrypted. The script I used (Figure 14) shows the scheme: the key is the SHA-256 hash of a passphrase, the Base64-decoded value begins with a 16-byte IV, and the rest is AES-CBC ciphertext with PKCS#7 padding. Knowing this pattern tells me what to hunt for: periodic GET requests with long Base64 query values, empty or near-empty responses to them, and HTML comments

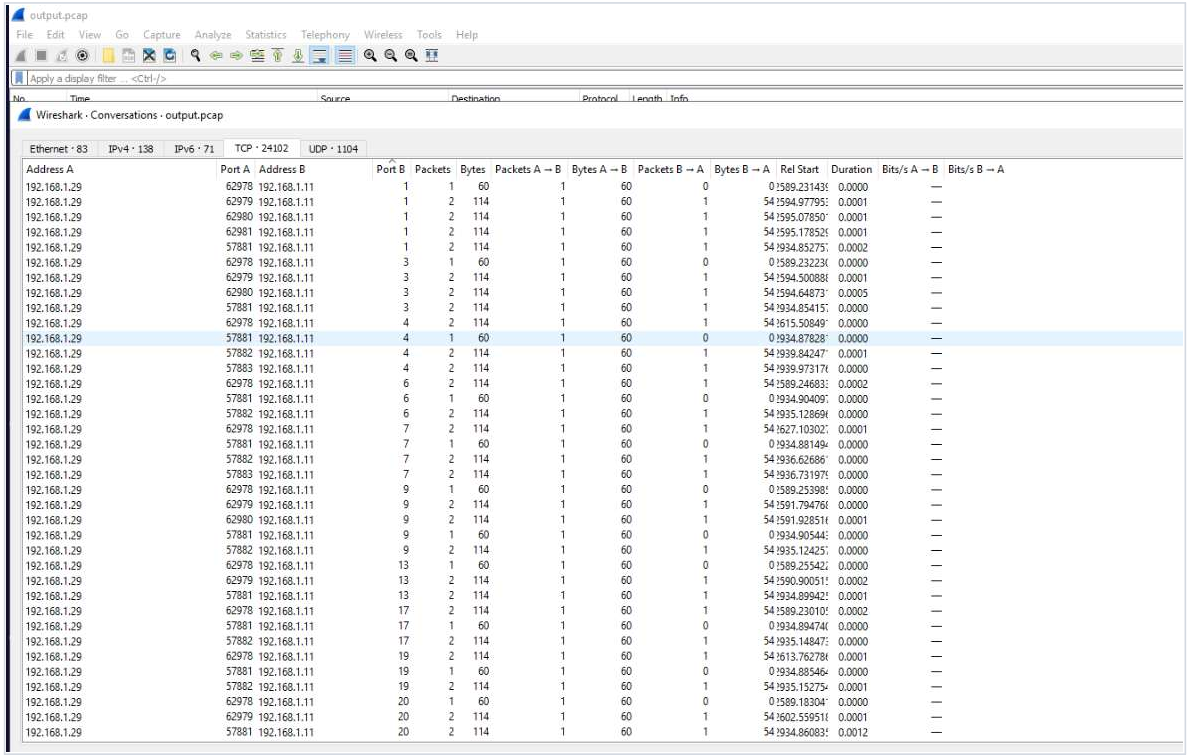
holding long random strings at the end of a page.

4. Investigation Phases

Phase 1 - Scoping the Capture and Spotting Reconnaissance

I started with Statistics > Conversations to see how the capture was composed. The tabs report 83 Ethernet, 138 IPv4, 71 IPv6, 24,102 TCP and 1,104 UDP conversations. The TCP count is far out of proportion to the number of hosts, so I sorted the TCP tab by the destination port.

Many conversations run from 192.168.1.29 to 192.168.1.11. Each has only one or two packets (a 60-byte probe and a short reply), lasts about 0.000 seconds, and targets the next destination port in sequence (1, 3, 4, 6, 7, 9, 13, 17, 19, 20 and so on) from a small set of source ports. This is the signature of a port scan of 192.168.1.11 by 192.168.1.29.



Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
192.168.1.29	62978	192.168.1.11	1	1	60	1	60	0	0	589.23143	0.0000	—	
192.168.1.29	62979	192.168.1.11	1	2	114	1	60	1	54	594.97795	0.0001	—	
192.168.1.29	62980	192.168.1.11	1	2	114	1	60	1	54	595.07850	0.0001	—	
192.168.1.29	62981	192.168.1.11	1	2	114	1	60	1	54	595.17852	0.0001	—	
192.168.1.29	57881	192.168.1.11	1	2	114	1	60	1	54	934.85275	0.0002	—	
192.168.1.29	62978	192.168.1.11	3	1	60	1	60	0	0	589.23223	0.0000	—	
192.168.1.29	62979	192.168.1.11	3	2	114	1	60	1	54	594.50088	0.0001	—	
192.168.1.29	62980	192.168.1.11	3	2	114	1	60	1	54	594.64873	0.0005	—	
192.168.1.29	57881	192.168.1.11	3	2	114	1	60	1	54	934.85415	0.0000	—	
192.168.1.29	62978	192.168.1.11	4	2	114	1	60	1	54	615.50849	0.0000	—	
192.168.1.29	57881	192.168.1.11	4	1	60	1	60	0	0	934.87828	0.0000	—	
192.168.1.29	57882	192.168.1.11	4	2	114	1	60	1	54	939.84247	0.0001	—	
192.168.1.29	57883	192.168.1.11	4	2	114	1	60	1	54	939.97317	0.0000	—	
192.168.1.29	62978	192.168.1.11	6	2	114	1	60	1	54	589.24683	0.0002	—	
192.168.1.29	57881	192.168.1.11	6	1	60	1	60	0	0	934.90409	0.0000	—	
192.168.1.29	57882	192.168.1.11	6	2	114	1	60	1	54	935.12869	0.0000	—	
192.168.1.29	62978	192.168.1.11	7	2	114	1	60	1	54	627.10302	0.0001	—	
192.168.1.29	57881	192.168.1.11	7	1	60	1	60	0	0	934.88149	0.0000	—	
192.168.1.29	57882	192.168.1.11	7	2	114	1	60	1	54	936.62686	0.0000	—	
192.168.1.29	57883	192.168.1.11	7	2	114	1	60	1	54	936.73197	0.0000	—	
192.168.1.29	62978	192.168.1.11	9	1	60	1	60	0	0	589.25398	0.0000	—	
192.168.1.29	62979	192.168.1.11	9	2	114	1	60	1	54	591.79476	0.0000	—	
192.168.1.29	62980	192.168.1.11	9	2	114	1	60	1	54	591.92851	0.0001	—	
192.168.1.29	57881	192.168.1.11	9	1	60	1	60	0	0	934.90544	0.0000	—	
192.168.1.29	57882	192.168.1.11	9	2	114	1	60	1	54	935.12425	0.0000	—	
192.168.1.29	62978	192.168.1.11	13	1	60	1	60	0	0	589.25542	0.0000	—	
192.168.1.29	62979	192.168.1.11	13	2	114	1	60	1	54	590.90051	0.0002	—	
192.168.1.29	57881	192.168.1.11	13	2	114	1	60	1	54	934.89942	0.0001	—	
192.168.1.29	62978	192.168.1.11	17	2	114	1	60	1	54	589.23010	0.0002	—	
192.168.1.29	57881	192.168.1.11	17	1	60	1	60	0	0	934.89474	0.0000	—	
192.168.1.29	57882	192.168.1.11	17	2	114	1	60	1	54	935.14847	0.0000	—	
192.168.1.29	62978	192.168.1.11	19	2	114	1	60	1	54	613.76278	0.0001	—	
192.168.1.29	57881	192.168.1.11	19	1	60	1	60	0	0	934.88546	0.0000	—	
192.168.1.29	57882	192.168.1.11	19	2	114	1	60	1	54	935.15275	0.0001	—	
192.168.1.29	62978	192.168.1.11	20	1	60	1	60	0	0	589.18304	0.0000	—	
192.168.1.29	62979	192.168.1.11	20	2	114	1	60	1	54	602.55951	0.0001	—	
192.168.1.29	57881	192.168.1.11	20	2	114	1	60	1	54	934.86083	0.0012	—	

Figure 1. Wireshark TCP conversations: short probes from 192.168.1.29 to 192.168.1.11 across consecutive low ports.

Phase 2 - Web Content Discovery Against the Workstation

Next I looked for failed web requests with the filter `http and ip.addr == 192.168.1.11 and http.response.code >= 400`. The workstation answers on port 80 with identical 523-byte `HTTP/1.0 404 File not found` responses. The first appears at 13:01:37, and at 13:44:57 a burst of them goes to 192.168.1.29, only milliseconds apart. A person browsing does not generate that pattern; it indicates an automated wordlist-based search for hidden files and directories.

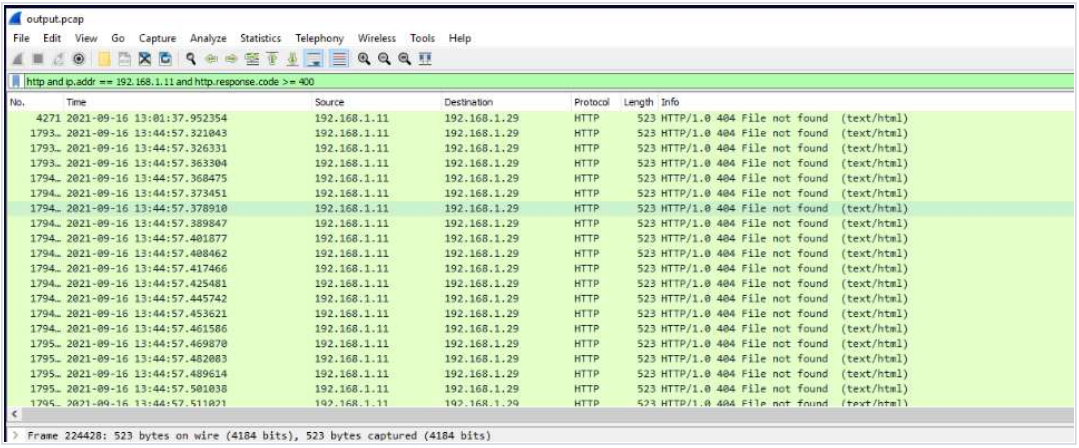


Figure 2. 404 responses from 192.168.1.11 to 192.168.1.29 (burst at 13:44:57).

Changing the destination shows a second burst at 14:15:25, this time to 192.168.1.37, with the same response size and rhythm. Two separate hosts therefore ran content discovery against the same web service.

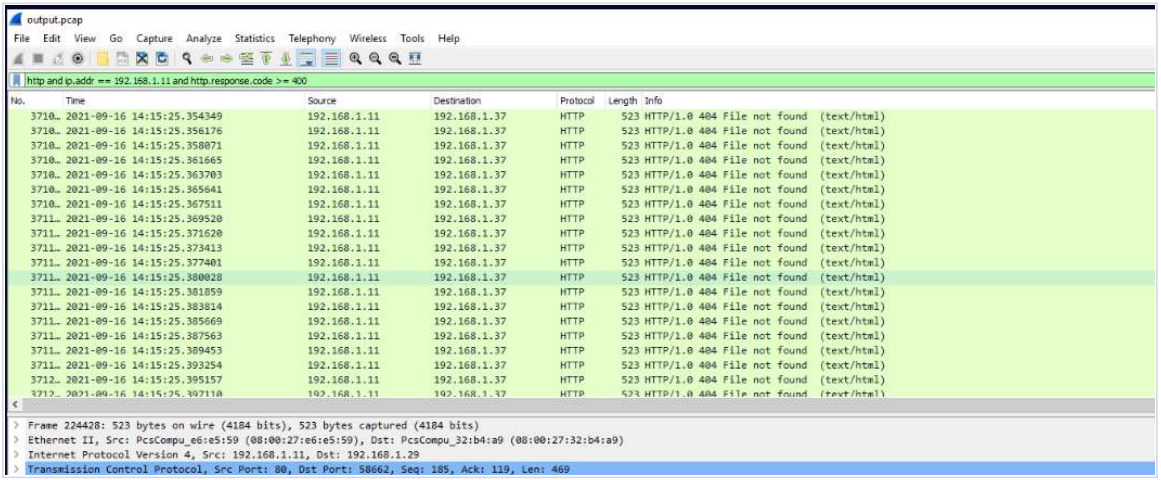


Figure 3. 404 responses from 192.168.1.11 to 192.168.1.37 (burst at 14:15:25).

Phase 3 - Finding the Beaconing Traffic

I then switched to what the workstation itself sends out, filtering on `http and ip.src == 192.168.1.11 and http.request.method == "GET" and http.request.full_uri contains "?"`. Apart from one request to 23.36.7.153 with a `Microsoft-WNS/10.0` user agent, which looks like ordinary Windows notification traffic, every hit is a `GET /images?guid=<long Base64 string>` request to 192.168.1.30.

These requests share several traits: the same Internet Explorer 11 style user agent (`Mozilla/5.0 (Windows NT 6.3; Trident/7.0; rv:11.0) like Gecko`), the same `Host: 192.168.1.30`, and the same static cookie `sessionid=nvWeAppsXasGYDJ`. The filtered list covers 13:49:31 to 14:17:34, the gaps between requests are irregular (from about ten seconds up to roughly sixteen minutes), and the frame lengths vary from a few hundred bytes to more than 8,000 bytes, consistent with command output of different sizes.

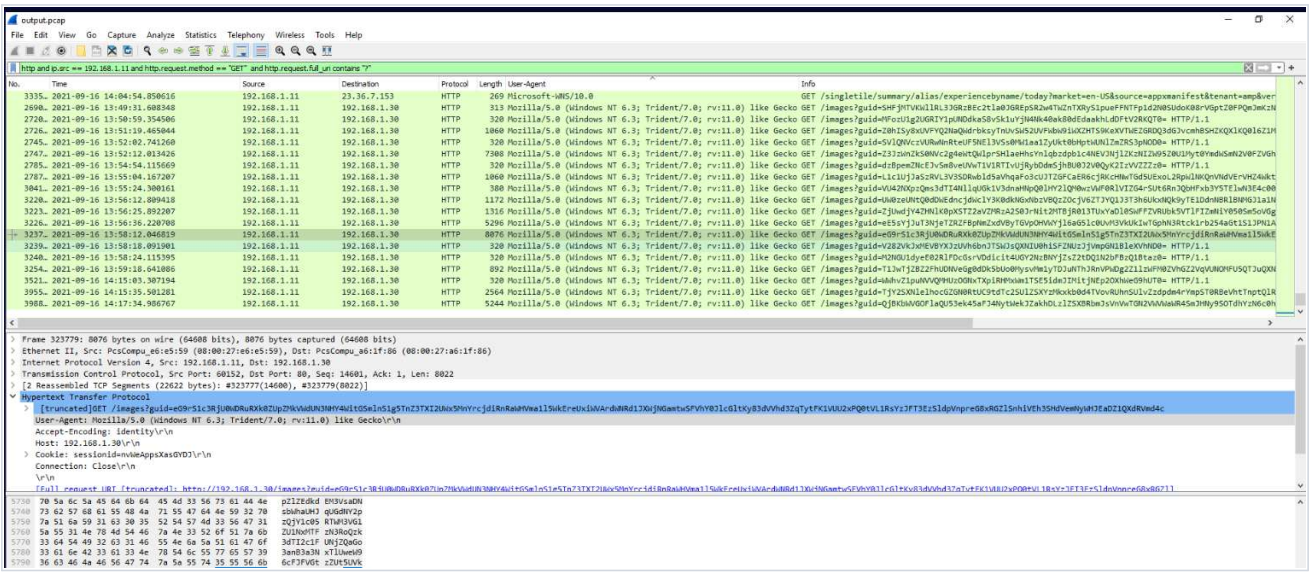


Figure 4. Beacon requests from 192.168.1.11 to 192.168.1.30 with the Base64 value visible in the guid parameter.

Restricting the view to `http and ip.src == 192.168.1.11 and ip.addr == 192.168.1.30` also reveals plain `GET / HTTP/1.1` requests (frame 269078) between the beacons. The client therefore alternates between polling the root page for a command and returning results through `/images?guid=`.



Figure 5. Traffic from the workstation to 192.168.1.30: guid beacons and root-page polling requests.

Phase 4 - Anatomy of One Exchange

Following TCP stream 14250 shows a beacon request with its full Base64 value. The server answers `HTTP/1.1 200 OK` with `Server: IIS`, a `Date` of Thu, 16 Sep 2021 13:51:18 GMT and `Content-Length: 0`. The data therefore travels in the request, and the answer is an empty acknowledgement.

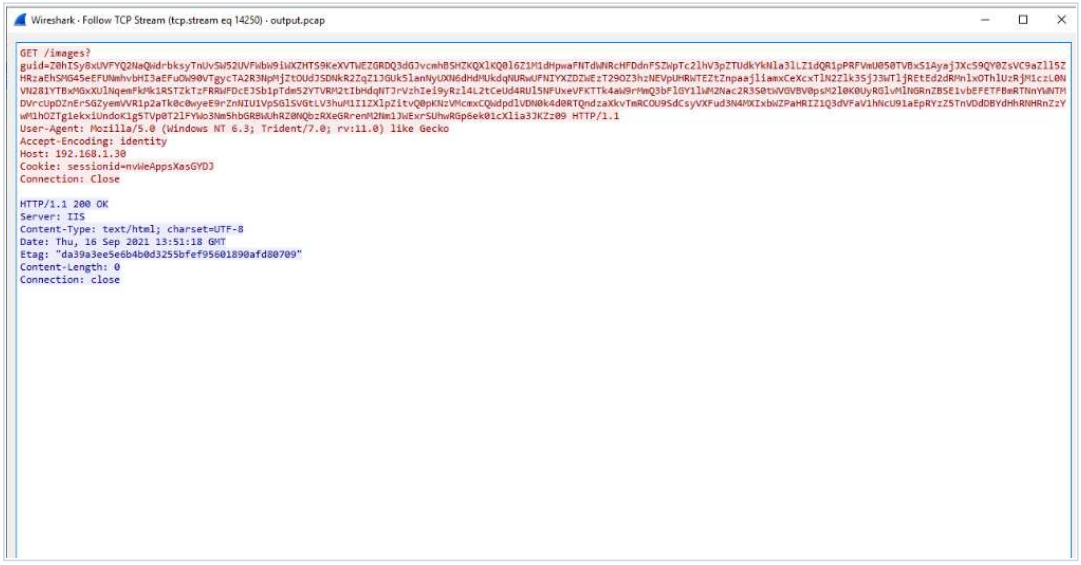


Figure 6. TCP stream 14250: a guid beacon answered with an empty 200 OK.

The next stream, 14251, is the interesting one. The response is a complete HTML page, a copy of a legitimate public web page with its inline scripts. The command is not in a visible element: it sits at the very end of the document, in an HTML comment just before `</body></html>`.

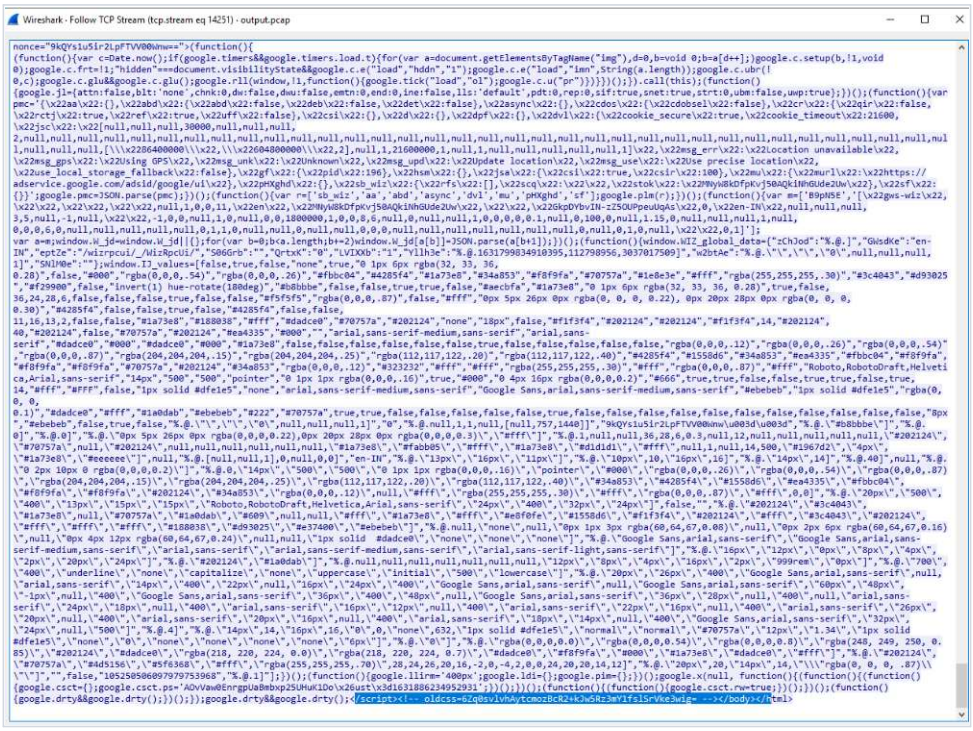


Figure 7. TCP stream 14251: a cloned web page returned by the C2 server.

Selecting the end of the stream (packet 274749, one client packet and 76 server packets) highlights the hidden value, a long random-looking string preceded by `oldcss=`. Its first characters are `qplSJcHhbKs1y9f`, which I use later to match it to a decrypted command.

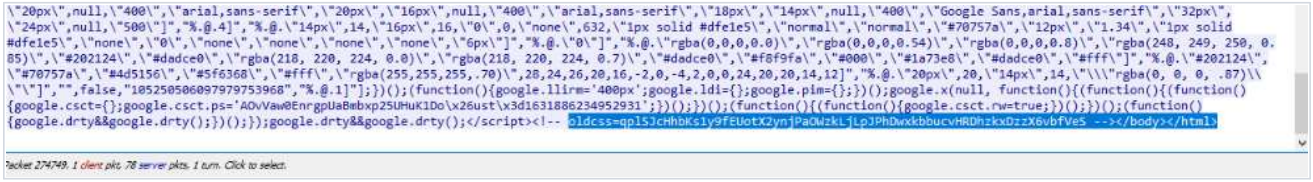


Figure 8. End of the cloned page: the highlighted oldcss comment carries the encrypted command.

Phase 5 - Recognising the Framework and Choosing a Decryption Method

The `oldcss=` comment at the end of a cloned page, together with query-string exfiltration over GET, matches TrevorC2. The passphrase used by the framework, `Tr3v0rC2R0x@nd1s@w350m3#TrevorForget`, even contains the tool's name. I first tried to decrypt a value with CyberChef's AES Decrypt operation (CBC mode, the passphrase as a UTF-8 key, no IV), but that operation expects a raw key and an explicit IV, whereas this implementation derives the key with SHA-256 and prepends the IV to the ciphertext. I therefore moved the work to a short Python script that follows the same steps:



Figure 9. CyberChef AES Decrypt configured with the passphrase; this operation does not match the framework's key derivation.

```
key = hashlib.sha256(passphrase.encode()).digest()
raw = base64.b64decode(value)
iv, ciphertext = raw[:16], raw[16:]
plain = unpad(AES.new(key, AES.MODE_CBC, iv).decrypt(ciphertext), 16)
```

Phase 6 - Decrypting the Commands

I copied the `oldcss` values from the successive server responses. The pieces below are the highlighted strings as they wrapped in Wireshark's stream view; the full values were used for decryption.

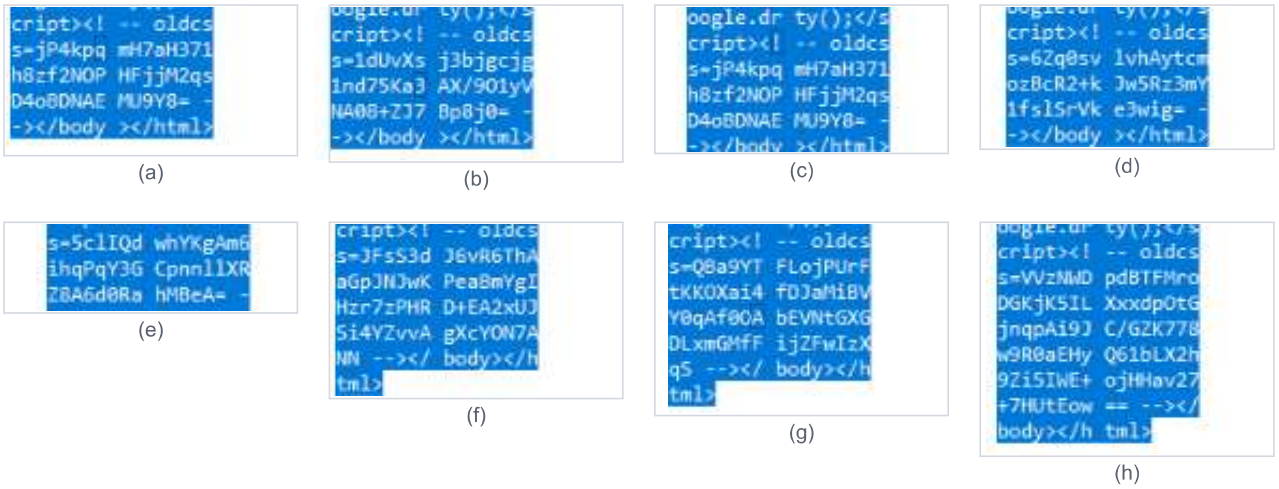


Figure 10. Fragments of oldcss values collected from successive C2 responses.

Running the first script on the value starting `JFsS3d` produced `MSEDGEWIN10:::cd`. The plaintext format is `<host name>:::<command>`, which also names the infected host.

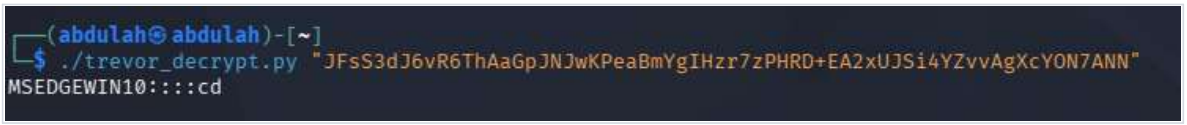


Figure 11. First decrypted value: the command cd for host MSEDGEWIN10.

The third value stopped the script with a UTF-8 decoding error (Figure 12). I revised the script so it tries UTF-8 and falls back to Latin-1 when the bytes are not valid UTF-8, and built an interactive version that decrypts one pasted value after another.



Figure 12. The first version of the script failing on the third value with a UTF-8 decoding error.

The interactive tool decrypted the five values in order. A first attempt on the last one failed with `Data must be padded to 16 byte boundary in CBC mode` because the pasted value was cut short; pasting the complete string succeeded.

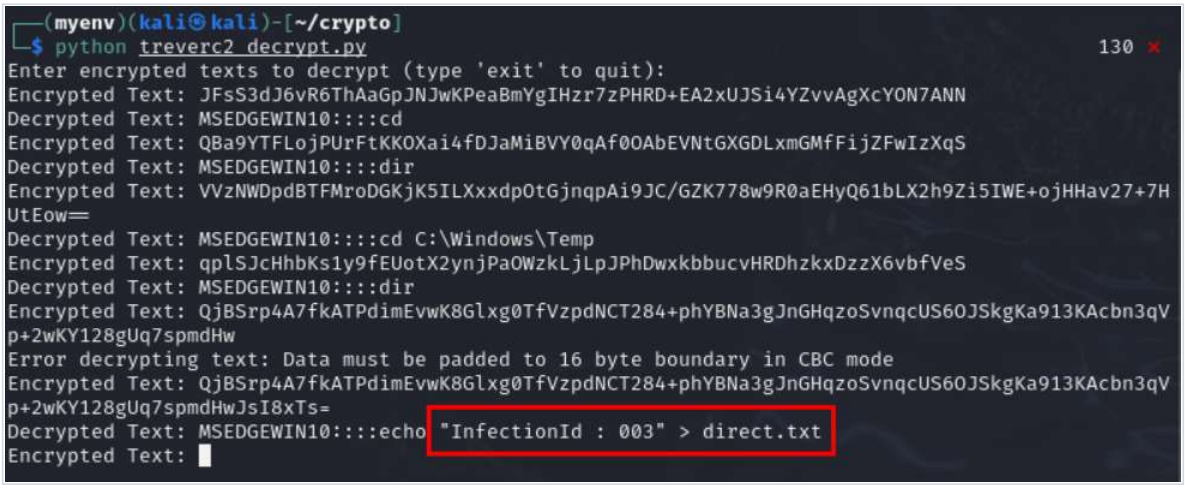


Figure 13. Interactive decryptor: the operator's commands in order, including the marker-file command (highlighted).

#	Encrypted value starts with	Decrypted command
1	`JFsS3d6vR6ThAaGp...`	`cd`
2	`QBa9YTFLojPUrFtK...`	`dir`
3	`VVzNWDpdBTFMroDG...`	`cd C:\Windows\Temp`
4	`qplSJcHhbKs1y9fE...`	`dir` (the value highlighted in Phase 4)
5	`QjBSrp4A7fkATPdi...`	`echo "InfectionId : 003" > direct.txt`
6	Later response (Figure 17)	`powershell -ep bypass -f exfil.ps1`

The source of the revised script is shown below. The comment in the code explains the Latin-1 fallback, and the main block holds the ciphertext under test.

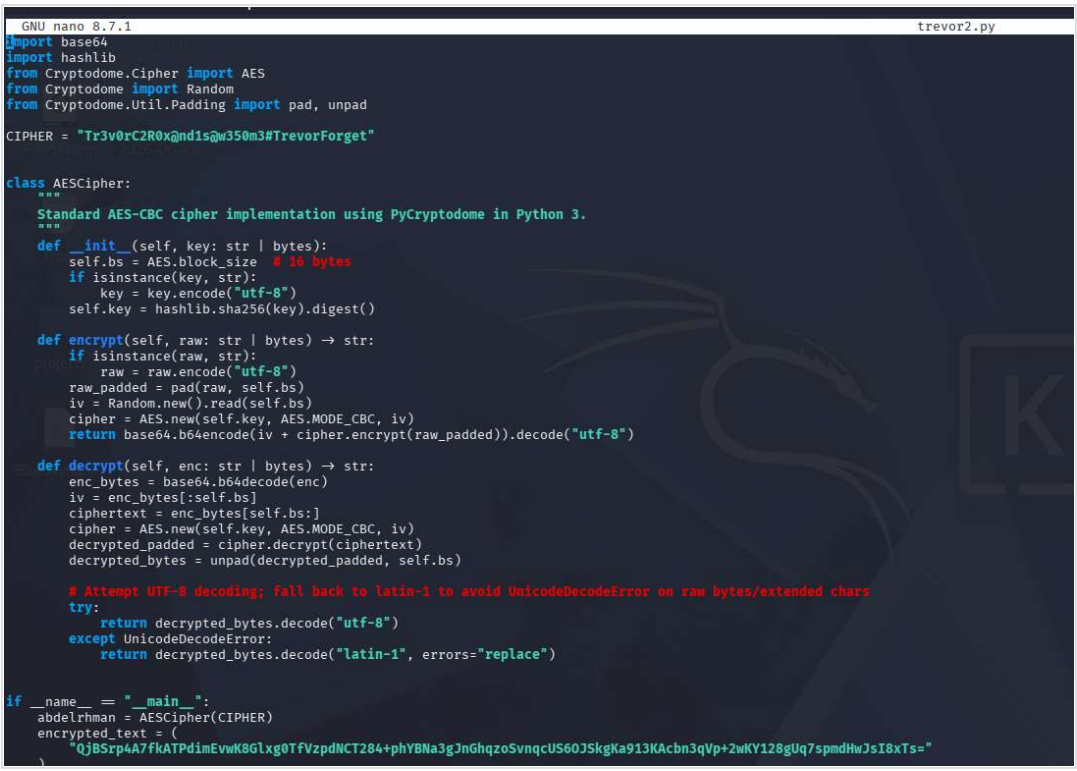


Figure 14. Decryption script: key derived with SHA-256, IV taken from the first 16 bytes, AES-CBC, UTF-8 with Latin-1 fallback.

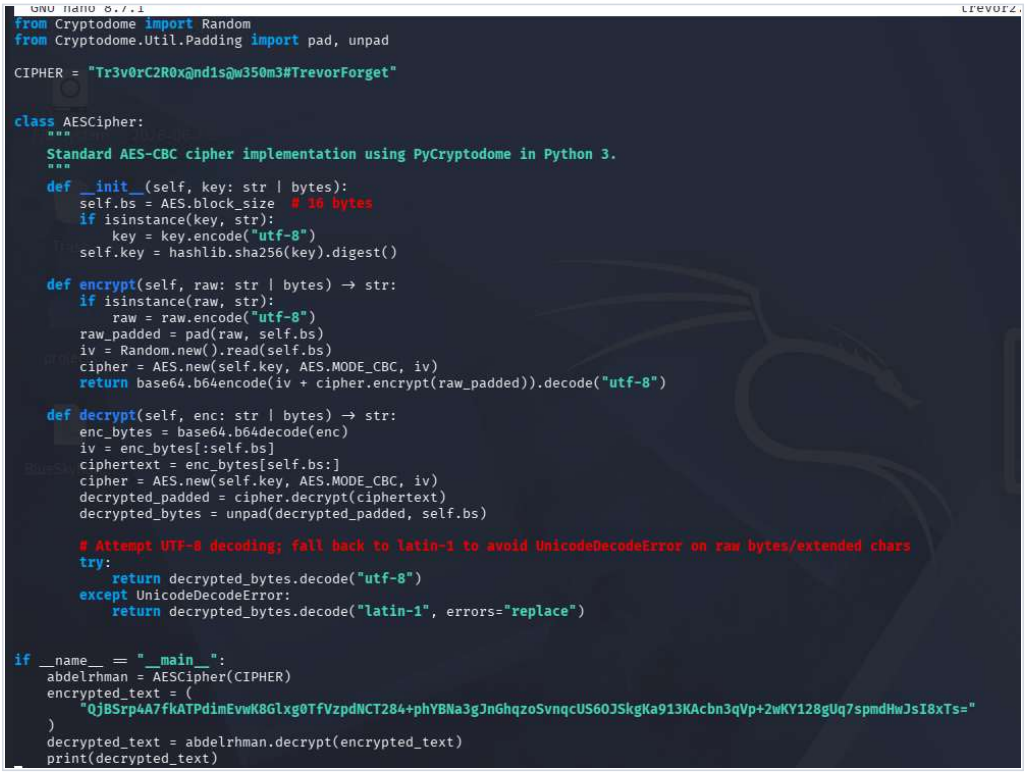


Figure 15. Main block of the script with the ciphertext of command 5 and the decrypt call.

Running that script against command 5 returned partly corrupted plaintext, although the tail `: 003 > direct.txt` was already readable; the interactive tool above returned the complete command, which writes `InfectionId : 003` into a file named `direct.txt`. This looks like a marker recording which infected host the operator is working on.



Figure 16. Script output for command 5, partly garbled but ending in : 003 > direct.txt.

I then collected further values from later responses and ran them through the same script.

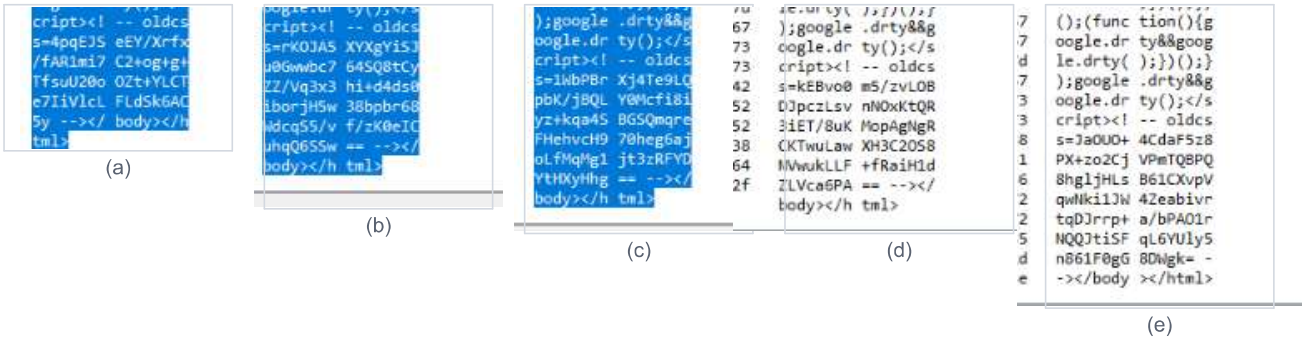


Figure 17. Fragments of the later oldcss values collected for decryption.

The last command recovered launches a PowerShell script with the execution policy bypassed. The script name suggests an exfiltration routine.



Figure 18. Decrypted later command: powershell -ep bypass -f exfil.ps1 for host MSEDGEWIN10.

Phase 7 - DNS Exfiltration

DNS traffic stood out in the capture. Filtering on `dns.flags.response == 0 and dns.qry.name contains "fakec2server.domain"` lists queries sent from an IPv6 link-local address, starting at 14:16:22. The name is an invented domain, so none of these queries can be legitimate. Each query has a Base64-looking first label, for example `ZQA=` or `bABpAA==`. Every label is asked four times (A, AAAA, A, AAAA with transaction IDs 0x0002 to 0x0005) about two seconds apart, a retry pattern typical of a resolver that is not getting an answer, since the data is carried in the name itself.

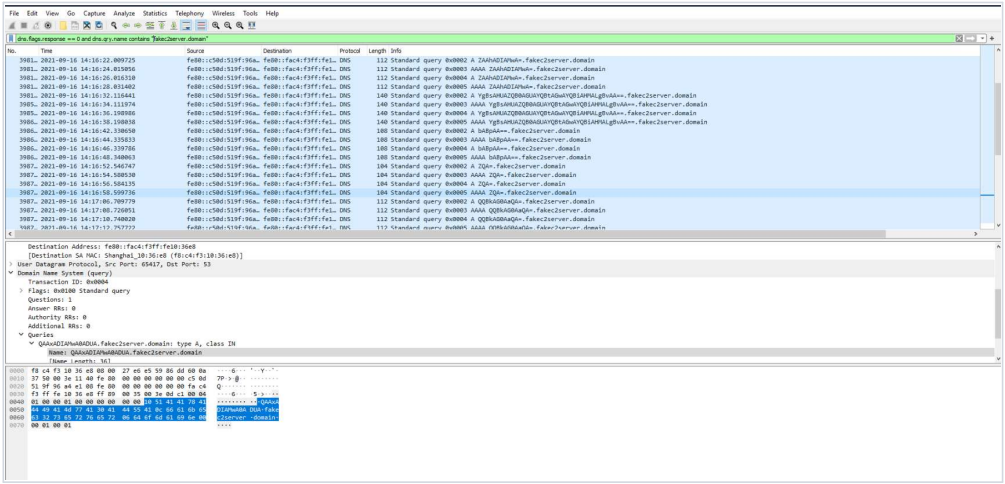


Figure 19. DNS queries with Base64 subdomains of fakec2server.domain, starting at 14:16:22.

In CyberChef I pasted the eight distinct query names and built a three-step recipe: Find / Replace (regex, global and multiline) removed `.fakec2server.domain`, From Base64 decoded the labels, and Remove null bytes dropped the zero bytes left by the UTF-16 encoding. The 46-character output is a password-style string followed by an e-mail style address, visible in Figure 20. Sensitive data was therefore leaving the network through DNS.

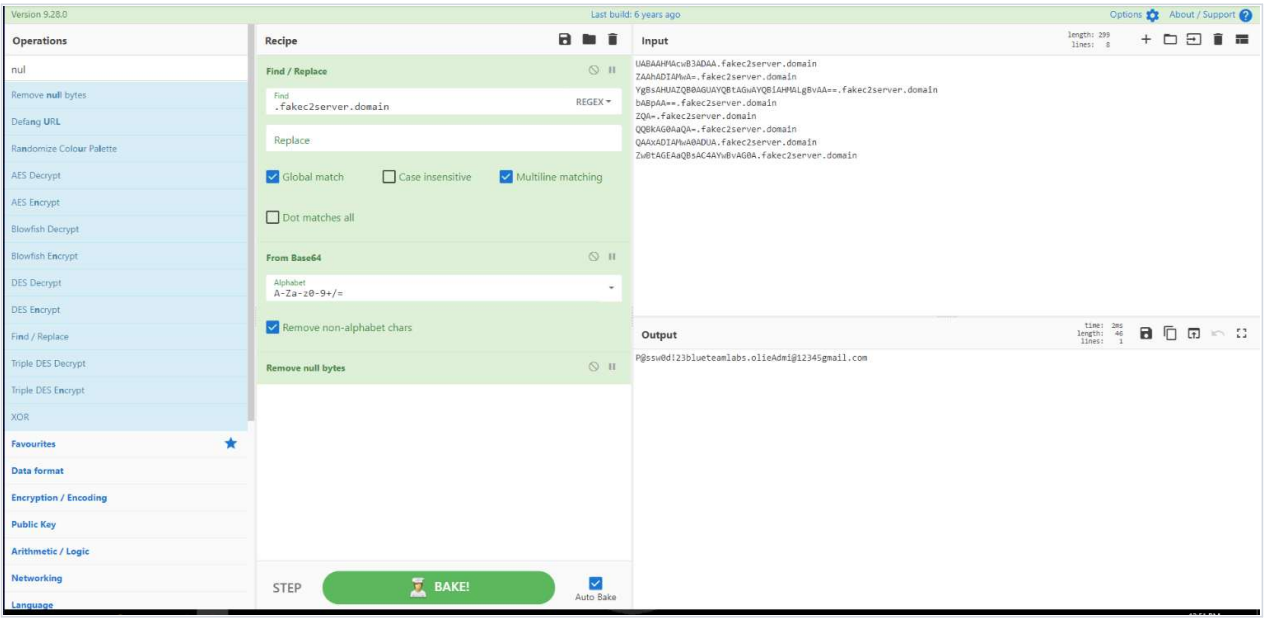


Figure 20. CyberChef recipe decoding the DNS labels into the exfiltrated text.

The DNS queries fall in the same minutes as the last C2 commands (the final beacon in my filtered view is at 14:17:34), so I assess them as related to the exfiltration step. The capture alone does not prove which process produced them.

5. Attack Timeline

Time (2021-09-16)	Event
Not shown	Port scan of 192.168.1.11 by 192.168.1.29 (conversation view, relative times only)
13:01:37	First 404 response from 192.168.1.11 to 192.168.1.29 in the filtered view
13:44:57	Burst of 404 responses to 192.168.1.29 (automated content discovery)
13:49:31	Earliest C2 beacon in the filtered view: GET /images?guid= to 192.168.1.30
13:51:18	Beacon in stream 14250 acknowledged with an empty 200 OK
14:15:25	Burst of 404 responses to 192.168.1.37 (second content discovery)
14:16:22	DNS queries with Base64 labels to fakec2server.domain begin
14:17:34	Last beacon visible in the filtered view

Times are taken from the Wireshark time column; the HTTP Date header (13:51:18 GMT) agrees with the packet times, so they are UTC.

6. MITRE ATT&CK Mapping

Technique	ID	Observed activity
Active Scanning	T1595	Sequential port probes from 192.168.1.29
Active Scanning: Wordlist Scanning	T1595.003	Bursts of 404 responses to automated path requests
Application Layer Protocol: Web Protocols	T1071.001	Beacons and commands carried in HTTP GET traffic
Data Obfuscation	T1001	Commands hidden in an HTML comment of a cloned page
Data Encoding: Standard Encoding	T1132.001	Base64 in the guid parameter and DNS labels
Encrypted Channel: Symmetric Cryptography	T1573.001	AES-CBC protected command and result values
Exfiltration Over C2 Channel	T1041	Command output returned in the guid query string
Exfiltration Over Alternative Protocol	T1048.003	Data encoded into DNS query names
Command and Scripting Interpreter: Windows Command Shell	T1059.003	cd, dir and echo commands
Command and Scripting Interpreter: PowerShell	T1059.001	powershell -ep bypass -f exfil.ps1
File and Directory Discovery	T1083	dir commands issued by the operator

7. Indicators of Compromise

Type	Value	Notes
IP address	192.168.1.30	C2 server receiving beacons
IP address	192.168.1.29	Port scan and content discovery source
IP address	192.168.1.37	Second content discovery source
Host	192.168.1.11 / MSEDGEWIN10	Compromised workstation
Domain	fakec2server.domain	Domain used for DNS exfiltration
URI	/images?guid=<Base64>	Beacon path carrying command output
HTTP cookie	sessionid=nvWeAppsXasGYDJ	Static cookie on every beacon
User agent	Mozilla/5.0 (Windows NT 6.3; Trident/7.0; rv:11.0) like Gecko	Used by the beacons
Page content	<!-- oldcss=... --> before </body></html>	Encrypted command carrier
Passphrase	Tr3v0rC2R0x@nd1s@w350m3#TrevorForget	Hard-coded AES passphrase
File names	direct.txt, exfil.ps1	Marker file and PowerShell script
Command	powershell -ep bypass -f exfil.ps1	Execution policy bypass

8. Impact Assessment

The beaconing channel was active for about half an hour (13:49 to 14:17) and gave the operator arbitrary command execution on MSEDGEWIN10 through traffic that blends in with web browsing. The infection marker suggests a tracked, deliberate intrusion. Credential-like data left the network through DNS, so the associated account should be considered exposed, and the host could serve as a foothold for movement to other systems.

9. Recommendations

- Isolate 192.168.1.11 and block 192.168.1.30 and fakec2server.domain at the network edge and the DNS resolver.
- Search the host for direct.txt and exfil.ps1, collect them for analysis, and rebuild the system from a known good state.
- Reset the password and any account that matches the data recovered from the DNS queries.
- Restrict the exposed web service on port 80, and investigate the hosts 192.168.1.29 and 192.168.1.37 that scanned it.
- Detect periodic GET requests with long Base64 query values, HTML comments of long random strings in responses, high-entropy DNS labels and repeated A/AAAA queries to unknown domains.
- Enable PowerShell script block logging and alert on the `-ep bypass` option.

10. Conclusion

The capture proves that 192.168.1.11 was controlled through a covert, encrypted C2 channel consistent with TrevorC2 and that data was exfiltrated over DNS. Decrypting the hidden values exposed the operator's commands, from simple enumeration to a PowerShell script launched with the execution policy bypassed. Isolation, credential rotation and the detections listed above are needed to contain the incident and to catch similar activity in future.